

Shell Scripting — Parte 1

1. Introdução ao Shell Script

1.1 O que é um shell script

Um shell script é um arquivo de texto contendo uma sequência de comandos do shell (interpretador de comandos) que são executados em lote, na ordem em que aparecem. No Ubuntu, o shell padrão é o Bash (Bourne Again SHell). Em vez de digitar comandos um a um no terminal, agrupamos esses comandos em um arquivo e pedimos ao sistema para executá-los automaticamente.

1.2 Por que aprender shell script

- Automatizar tarefas repetitivas de administração de sistemas (backups, limpeza de logs, criação de usuários).
- Orquestrar outros programas e utilitários de linha de comando (grep, awk, sed, curl).
- Criar rotinas de instalação, configuração e monitoramento de servidores.
- Base essencial para DevOps, CI/CD e administração de redes/sistemas Linux.

1.3 O interpretador e o shebang

A primeira linha de um script normalmente indica qual interpretador deve executá-lo. Essa linha é chamada de shebang (ou hashbang):

```
#!/bin/bash  
  
echo "Olá, mundo!"
```

O shebang começa obrigatoriamente na primeira linha, sem espaços antes de '#!'. Ele diz ao sistema operacional: "use o programa /bin/bash para interpretar este arquivo".

1.4 Criando, permitindo execução e executando um script

Passos básicos para criar e rodar um script chamado boasvindas.sh:

```
$ nano boasvindas.sh      # cria/edita o arquivo  
$ chmod +x boasvindas.sh  # dá permissão de execução  
$ ./boasvindas.sh         # executa o script  
  
# Alternativa sem permissão de execução:  
$ bash boasvindas.sh
```

💡 *chmod +x adiciona a permissão de execução (x) ao arquivo. Sem ela, o comando ./script.sh resulta em 'Permission denied'.*

1.5 Comentários

Tudo após o caractere # (exceto no shebang) é tratado como comentário e ignorado pelo interpretador:

```
#!/bin/bash
# Este script exibe a data e hora atuais
date    # comando que imprime a data
```

2. Variáveis

2.1 Declaração e atribuição

Em Bash, variáveis não têm tipo declarado explicitamente e a atribuição NÃO pode ter espaços ao redor do sinal de igual:

```
nome="Nelson"
idade=53
linguagem='Shell Script'

# ERRADO – gera erro de sintaxe:
nome = "Nelson"
```

2.2 Convenções de nomenclatura

- Iniciar com letra ou underscore (`_`), nunca com número.
- Usar apenas letras, números e underscore (sem espaços ou acentos).
- Por convenção, variáveis de ambiente/globais em MAIÚSCULAS e variáveis locais do script em minúsculas.

2.3 Acessando o valor de uma variável

Para ler o valor, usa-se o cifrão (\$) antes do nome, opcionalmente com chaves {}:

```
#!/bin/bash
nome="StackUp"
echo $nome
echo ${nome}
echo "Empresa: ${nome}Ltda" # chaves evitam ambiguidade ao concatenar
```

💡 Use sempre `${variavel}` quando for concatenar a variável colada a outro texto, para o shell saber onde o nome termina.

2.4 Variáveis de ambiente comuns

Variável	Significado
\$HOME	Diretório pessoal do usuário
\$USER	Nome do usuário logado
\$PWD	Diretório de trabalho atual
\$PATH	Lista de diretórios de busca de executáveis
\$SHELL	Shell padrão do usuário
\$HOSTNAME	Nome da máquina

2.5 Escopo: variáveis locais e export

Por padrão, uma variável criada em um script fica visível apenas dentro dele (e de sub-shells que ele herda). Para exportá-la ao ambiente, de forma que outros processos filhos também a enxerguem, usa-se export:

```
#!/bin/bash
SISTEMA="NetSafe"
export SISTEMA
# ou em uma linha:
export VERSAO="1.0"
```

2.6 Variáveis somente leitura

```
readonly PI=3.14159
PI=3.14 # erro: PI: readonly variable
```

2.7 Aspas simples, duplas e substituição de comando

Sintaxe	Comportamento
'texto'	Aspas simples: literal, NÃO expande variáveis nem comandos
"texto"	Aspas duplas: expande variáveis (\$var) e comandos
\$(comando)	Substituição de comando (forma moderna, preferida)
`comando`	Substituição de comando (forma antiga, crase)

```
#!/bin/bash
usuario='$USER'
echo "$usuario" # imprime literalmente $USER

usuario="$USER"
echo "$usuario" # imprime o usuário logado

data_atual=$(date +%d/%m/%Y)
echo "Hoje é $data_atual"
```

3. Entrada e Saída

3.1 Saída de dados: echo e printf

```
echo "Processando backup..."
echo -n "Sem quebra de linha ao final"
echo -e "Linha1\nLinha2\tTabulado" # -e habilita códigos de escape

printf "Nome: %s | Idade: %d\n" "Nelson" 53
printf "%-10s|%5d\n" "item" 42 # formatação de largura de campo
```

💡 *printf oferece controle mais preciso de formatação (como em C) e não adiciona quebra de linha automaticamente.*

3.2 Entrada de dados: read

```
#!/bin/bash
read -p "Digite seu nome: " nome
echo "Olá, $nome!"

read -sp "Digite sua senha: " senha # -s oculta a digitação
echo

read -t 5 -p "Você tem 5s para responder: " resp # -t define timeout

read -a lista -p "Digite 3 números separados por espaço: "
echo "Soma dos dois primeiros: $(( ${lista[0]} + ${lista[1]} ))"
```

3.3 Argumentos de linha de comando

Um script pode receber parâmetros ao ser chamado. Eles ficam disponíveis nas variáveis posicionais:

Variável	Significado
\$0	Nome do script
\$1, \$2 ...	Primeiro, segundo... argumento
\$#	Quantidade de argumentos recebidos
\$@	Todos os argumentos, cada um como palavra separada
\$*	Todos os argumentos como uma única string

```
#!/bin/bash
# uso: ./saudacao.sh Nelson Fatec
echo "Script: $0"
echo "Total de argumentos: $#"
```

```
echo "Primeiro: $1 | Segundo: $2"
```

```
echo "Todos: $@"
```

3.4 Redirecionamento

Operador	Efeito
>	Redireciona saída padrão para arquivo (sobrescreve)
>>	Redireciona saída padrão para arquivo (anexa)
<	Redireciona entrada padrão a partir de um arquivo
2>	Redireciona saída de erro
2>&1	Redireciona erro para o mesmo destino da saída padrão
&>	Redireciona saída padrão e erro juntos

```
ls -l /etc > listagem.txt
echo "Backup concluído" >> log.txt
comando_invalido 2> erros.log
comando_invalido > saida.log 2>&1
wc -l < arquivo.txt
```

3.5 Pipes

O pipe (|) conecta a saída de um comando à entrada do próximo, permitindo combinar utilitários:

```
ps aux | grep bash
cat /etc/passwd | cut -d: -f1 | sort
ls -l /var/log | wc -l
```

4. Operadores

4.1 Operadores aritméticos

Bash não faz aritmética por padrão em variáveis; é preciso usar `expr`, `let` ou a construção `$(( ))` (mais usada hoje):

```
a=10
b=3

soma=$((a + b))
sub=$((a - b))
mul=$((a * b))
div=$((a / b))    # divisão inteira
resto=$((a % b))
pot=$((a ** 2))

echo "Soma: $soma | Resto: $resto | Potência: $pot"

let contador=0
let "contador += 1"

resultado=$(expr $a + $b)  # forma antiga, exige espaços
```

💡 Para cálculos com casas decimais, o Bash não suporta ponto flutuante nativamente — use `bc`:
`echo "scale=2; 10/3" | bc`

4.2 Operadores relacionais (comparação)

Números e strings usam operadores diferentes dentro de `[ ]` (test):

Numéricos	Strings	Significado
<code>-eq</code>	<code>=</code> ou <code>==</code>	igual
<code>-ne</code>	<code>!=</code>	diferente
<code>-gt</code>	<code>></code>	maior que (dentro de <code>[[]]</code>)
<code>-lt</code>	<code><</code>	menor que (dentro de <code>[[]]</code>)
<code>-ge</code>	<code>-</code>	maior ou igual
<code>-le</code>	<code>-</code>	menor ou igual

```
a=5; b=8
if [ $a -lt $b ]; then echo "$a é menor que $b"; fi

nome="admin"
if [ "$nome" = "admin" ]; then echo "Acesso liberado"; fi
```

4.3 Operadores lógicos

Operador	Significado
&&	E lógico (executa o segundo comando só se o primeiro for bem-sucedido)
	OU lógico (executa o segundo comando só se o primeiro falhar)
!	Negação

```
mkdir backup && echo "Diretório criado com sucesso"
ping -c1 servidor || echo "Servidor inacessível"

idade=20
if [ $idade -ge 18 ] && [ $idade -le 65 ]; then
    echo "Idade dentro da faixa permitida"
fi
```

4.4 Operadores de teste de arquivo

Operador	Verifica
-e	o arquivo/diretório existe
-f	é um arquivo regular
-d	é um diretório
-x	tem permissão de execução
-r / -w	tem permissão de leitura / escrita
-s	existe e não está vazio

```
if [ -f "relatorio.txt" ]; then
    echo "Arquivo encontrado"
else
    echo "Arquivo não existe"
fi
```


5. Estudo de Caso ABP — NetSafe Ltda

A NetSafe Ltda é uma empresa fictícia de segurança de redes usada em nossas atividades práticas. Ela precisa de um script simples que colete dados de um técnico plantonista e gere um registro de atendimento.

```
#!/bin/bash
# registro_atendimento.sh - NetSafe Ltda

echo "=== NetSafe Ltda - Registro de Atendimento ==="
read -p "Nome do técnico: " tecnico
read -p "Número do chamado: " chamado
read -p "Cliente atendido: " cliente
data=$(date +%d/%m/%Y\ %H:%M)

arquivo="atendimento_${chamado}.log"

{
    echo "Técnico: $tecnico"
    echo "Chamado: $chamado"
    echo "Cliente: $cliente"
    echo "Data/Hora: $data"
} > "$arquivo"

if [ -s "$arquivo" ]; then
    echo "Registro salvo em $arquivo"
else
    echo "Falha ao salvar o registro"
fi
```

6. Exercícios Propostos

Contexto: você foi contratado(a) como estagiário(a) de suporte na StackUp Ltda, uma empresa fictícia de desenvolvimento de software. Resolva os exercícios abaixo criando scripts .sh reais e testando-os no Ubuntu.

Nível básico

1. Crie um script apresentacao.sh que declare variáveis com seu nome, curso e turma, e as exiba usando echo com \${}.
2. Crie um script calculadora.sh que leia dois números com read e exiba soma, subtração, multiplicação e divisão usando \$(()).
3. Crie um script argumentos.sh que receba 3 argumentos na chamada e exiba quantos foram passados (\$#) e cada um deles.

Nível intermediário

1. Crie um script login.sh que peça usuário e senha (senha oculta com read -s) e, usando operadores relacionais, verifique se usuário é 'stackup' e senha é '1234', exibindo mensagem de acesso permitido ou negado.
2. Crie um script verifica_arquivo.sh que receba um caminho de arquivo como argumento (\$1) e informe, usando operadores de teste de arquivo, se ele existe, é um diretório e se tem permissão de escrita.
3. Crie um script log_sistema.sh que redirecione a saída do comando 'df -h' para um arquivo relatorio_disco.txt e, em seguida, anexe (>>) a data atual ao final do arquivo.

Desafio

1. Crie um script cadastro_funcionario.sh para a StackUp Ltda que colete nome, cargo e salário-base via read, calcule o salário com um bônus de 10% usando bc (ponto flutuante), e grave tudo em um arquivo usando redirecionamento, no formato: 'Nome | Cargo | Salário Final'.

Referências

- GNU Bash Reference Manual — <https://www.gnu.org/software/bash/manual/>
- Ubuntu Server Guide — Scripting — <https://ubuntu.com/server/docs>
- Advanced Bash-Scripting Guide (TLDP)
- `man bash` (documentação local do sistema)